

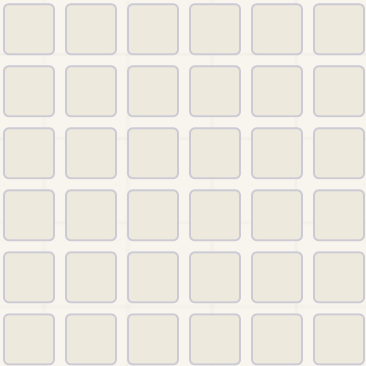
Aula 1

Transformers:

Introdução

Um percurso integral pelos fundamentos — de tarefas de NLP e tokenização até a arquitetura completa do transformer

DO ZERO À ARQUITETURA COMPLETA VASWANI ET AL., 2017



O caminho completo

Contexto de NLP → representações → RNNs → nascimento da atenção → arquitetura completa do transformer

01	NLP overview	tarefas & métricas	07	Transformer architecture	encoder & decoder
02	Tokenização	word · subword · char	08	Input & positional encoding	V, d_model
03	Word representation	one-hot → embeddings	09	Encoder & Decoder	N, h, d_FF, d_key, d_value
04	RNNs & LSTM	e suas limitações			
05	Nascimento da atenção	Bahdanau, 2014			
06	Self-attention mechanism	Q, K, V			

Três famílias de problemas

CLASSIFICAÇÃO

texto → 1 rótulo

- Sentiment extraction
- Intent detection
- Language detection
- Topic modeling

accuracy · precision · recall · F1

"MULTI"-CLASSIFICAÇÃO

texto → 1 rótulo por token

- Part-of-speech tagging
- Named entity recognition
- Dependency parsing
- Constituency parsing

precision/recall por entidade

GERAÇÃO

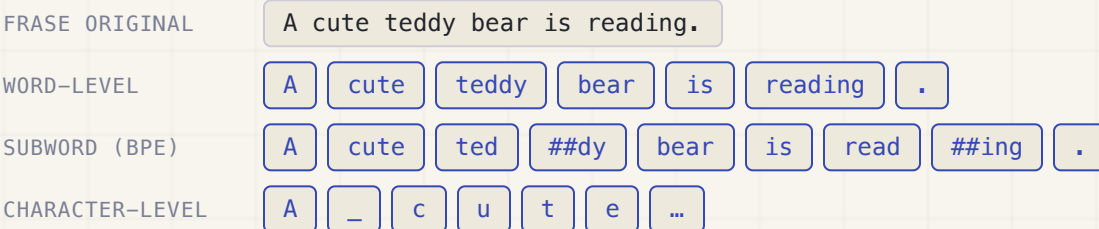
texto → texto

- Tradução automática
- Question answering
- Sumarização
- Geração de texto

BLEU · ROUGE · perplexidade

Como quebrar o texto em unidades

Essa escolha determina o vocabulário V e volta quando falarmos do parâmetro $\mathbf{V}$ na entrada e saída do transformer.



MÉTODO	VANTAGENS	DESVANTAGENS
Word-level	Simples, interpretável	Risco de OOV; não aproveita raízes
Subword	Aproveita prefixos/sufixos comuns; aprendido dos dados	Ainda há risco de OOV, porém menor
Character-level	Chance mínima de OOV; robusto a erros de digitação	Sequências mais longas; embeddings menos interpretáveis

De rótulos arbitrários a vetores

Um token precisa virar número antes de entrar em qualquer rede. A escolha de representação afeta diretamente o que o modelo consegue aprender.

ONE-HOT ENCODING

Cada token é um vetor de tamanho **V** (vocabulário) com um único 1. Não carrega noção de similaridade: "gato" e "gata" são tão distantes quanto "gato" e "carro".

EMBEDDING APRENDIDO

Vetor denso de dimensão **d** $\ll$ **V**, aprendido a partir dos dados. Palavras com uso similar terminam próximas no espaço vetorial.

Word2Vec (Mikolov et al., 2013)

Rede rasa treinada numa tarefa-proxy sobre bilhões de palavras. O que sobra de útil é a camada oculta: o embedding. Mas cada palavra ainda tem **um único vetor fixo** — sem contexto.



CBOW

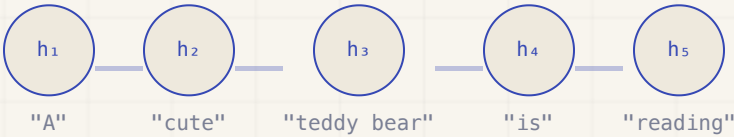
Prevê a palavra central a partir do contexto: "...A cute **teddy bear** is reading..." → contexto prevê "teddy bear".

SKIP-GRAM

O inverso: da palavra central, prevê o contexto ao redor. Mais eficaz com palavras raras.

Carregando um estado adiante

Introduzida nos anos 1980. Cada estado oculto h_t depende de $h_{(t-1)}$ e do token atual — isso captura ordem, o que Word2Vec ignorava.



LSTM (1997) — "Long Short-Term Memory": estrutura mais elaborada no estado da célula (gates de entrada/esquecimento/saída), mitigando — mas não eliminando — o problema seguinte.

Duas rachaduras estruturais

São essas duas rachaduras que motivam, primeiro a atenção (2014) e depois o transformer inteiro (2017).

01 Vanishing gradient. Ao retropropagar por muitos passos temporais, o gradiente encolhe exponencialmente — dependências de longo alcance ficam quase invisíveis para o treinamento.

02 Computação estritamente sequencial. h_t não pode ser calculado antes de $h_{(t-1)}$ — isso impede paralelização em GPU.

O primeiro remendo: Bahdanau et al.

Em tradução (seq2seq), o encoder precisava espremer a frase inteira em um único vetor de contexto — frases longas eram esquecidas. A solução: deixar o decoder "olhar de volta" para partes específicas do input.

A cute teddy bear is reading

‡ alinhamento aprendido, não posicional

Un ours en peluche mignon lit

"ours en peluche" (3 tokens em francês) alinha com "teddy bear" (2 tokens em inglês) — o alinhamento é aprendido a partir do conteúdo, não da posição. É o embrião conceitual de query/key/value.

Overview do Transformer

Introduzido no paper "Attention Is All You Need" — depende inteiramente do mecanismo de self-attention, sem nenhuma recorrência. Resultados state-of-the-art em tarefas de tradução automática.

VASWANI ET AL., 2017

0

camadas recorrentes — self-attention substitui a recorrência por completo

$O(1)$

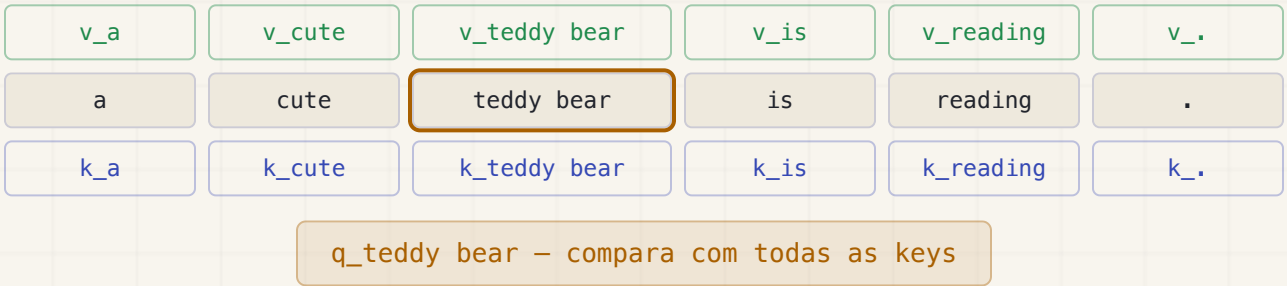
passos sequenciais entre duas posições quaisquer

100%

paralelizável em GPU via forma matricial

O conceito de Query, Key, Value

Cada token gera três vetores. A query de um token é comparada com a key de todos os outros — o resultado pondera os values na soma final.



Computação eficiente com matrizes

A mesma ideia, vetorizada — é isso que roda de fato em GPU, processando todos os tokens de uma vez.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Q, K, V empilhados como matrizes — uma linha por token. A saída é uma média ponderada dos values.

As três peças da arquitetura

Encoder e decoder são pilhas repetidas dessas três peças. Vamos percorrer cada bloco do diagrama nos próximos slides.

ATTENTION LAYER (MHA)

Self-attention (encoder-encoder, decoder-decoder) + encoder-decoder attention

FEED FORWARD NN (FFNN)

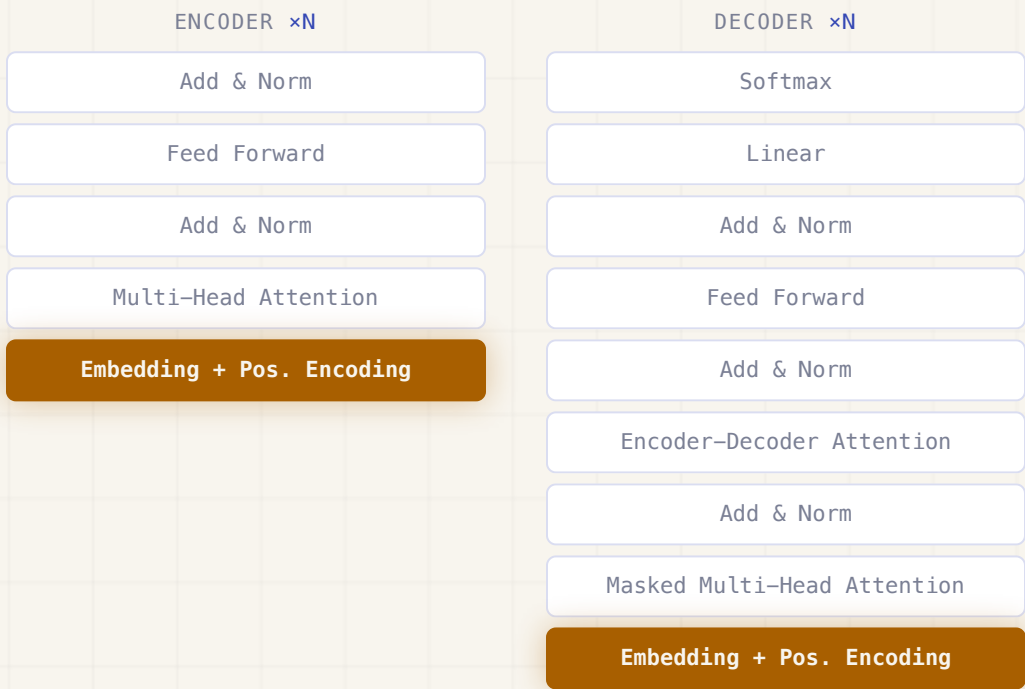
Aplicada independentemente a cada posição/token

POSITIONAL ENCODING (PE)

Injeta informação de posição, já que self-attention por si só é invariante à ordem

Da entrada bruta ao embedding

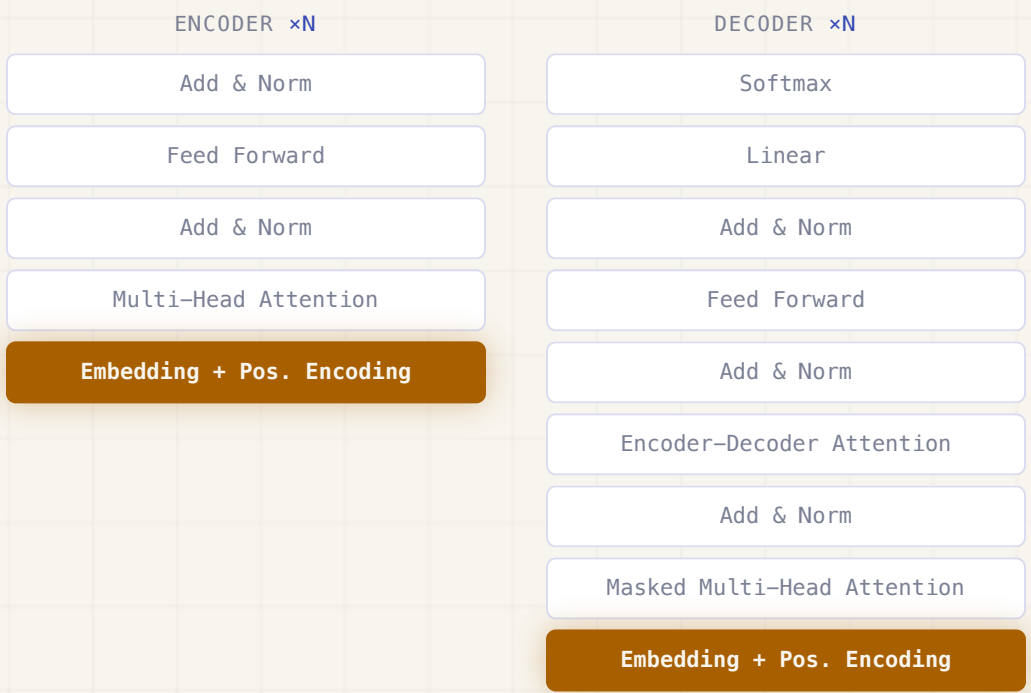
Texto é tokenizado, cada token vira um embedding aprendido.



V – tamanho do vocabulário d_{model} – dimensão do embedding

...com um truque

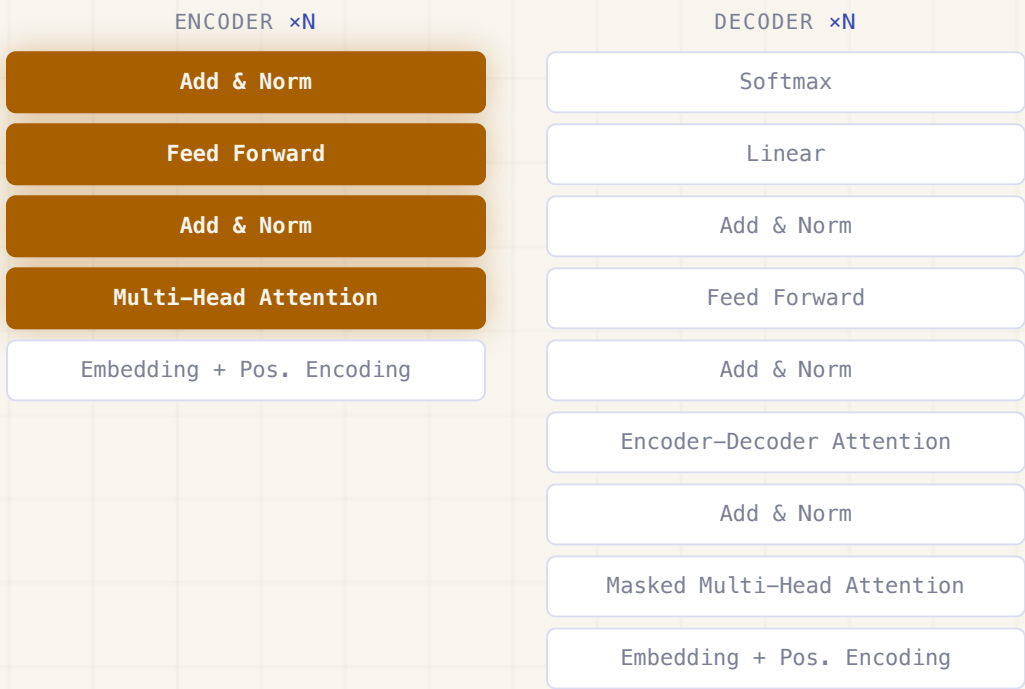
Self-attention é invariante à ordem — sem PE, "o cachorro mordeu o homem" e "o homem mordeu o cachorro" seriam idênticos para o modelo. Solução: somar um padrão de posição aos embeddings, aprendido ou fixo (senoidal).



Objetivo: deixar o modelo entender posição **relativa** entre tokens. Pode ser aprendido ou hardcoded (padrão senoidal do paper original).

Encoder — self-attention + FFN

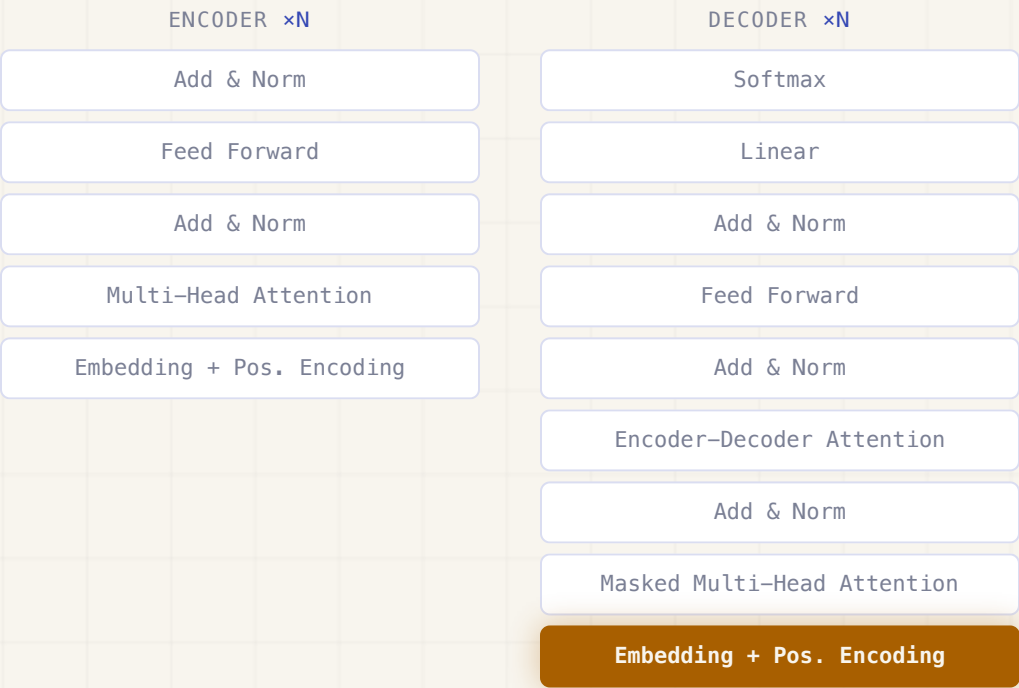
Encoder-encoder attention (self-attention) + feed forward + camada de normalização, empilhados N vezes.



N — camadas empilhadas h — número de heads $d_{FF}, d_{key}, d_{value}$ — dimensões das sub-camadas d_{model} — dimensão do embedding

Output "shifted right"

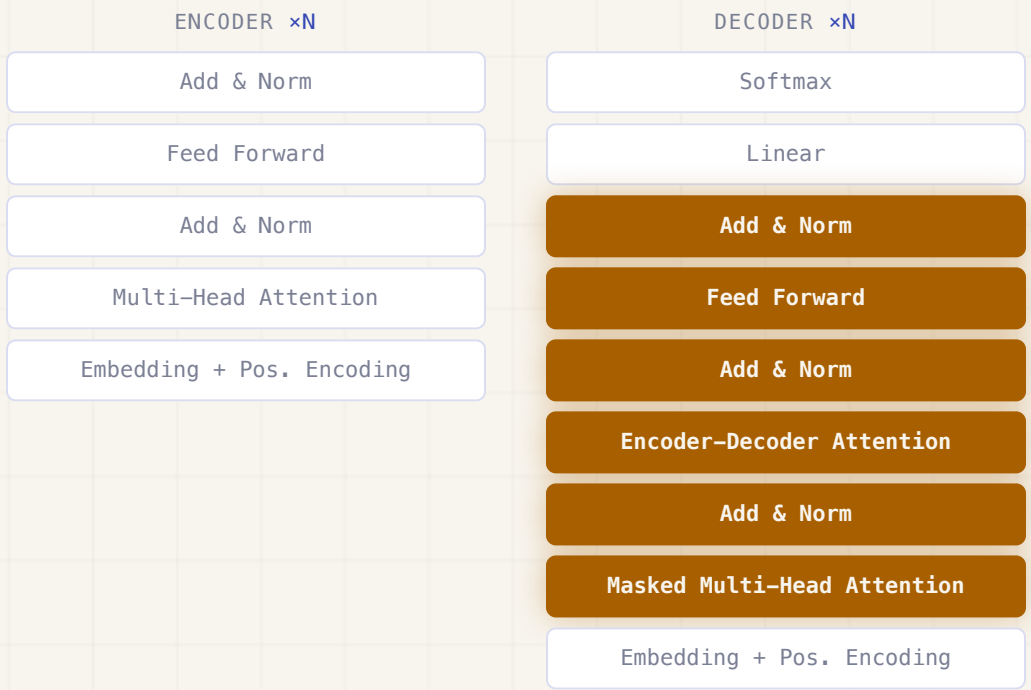
O decoder recebe embeddings aprendidos dos tokens de saída — na prática, começa apenas com o token especial [BOS] durante a tradução.



V – tamanho do vocabulário d_{model} – dimensão do embedding

Decoder — self, cross e FFN

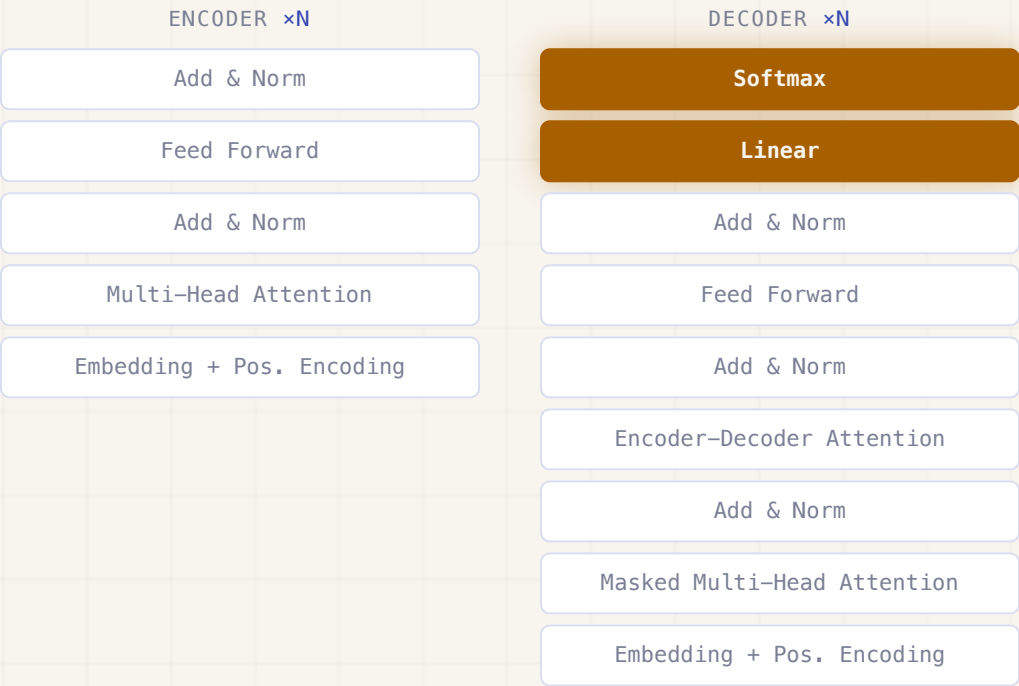
Decoder-decoder attention (mascarada) + encoder-decoder attention (cross-attention) + feed forward + normalização, empilhados N vezes.



N – camadas empilhadas **h** – número de heads **d_FF, d_key, d_value** – dimensões das sub-camadas **d_model** – dimensão do embedding

Output — linear + softmax

Projeção linear seguida de softmax: um problema de classificação onde cada classe é uma palavra do vocabulário.



V – tamanho do vocabulário d_{model} – dimensão do embedding